

C o I

Composition over Inheritance Principle

Has-a is more flexible than is-a

DEFINITION

Reuse behavior by composing small objects rather than inheriting from base classes.

Inheritance binds a subclass to its parent at compile time and exposes the parent's internals, so behavior is fixed and reuse is rigid. Composition gives an object collaborators it talks to through narrow interfaces, wired at runtime, so behavior can vary and swap. "Favor composition" does not ban inheritance — it reserves it for a genuine is-a with a shared, stable contract, and uses has-a for everything you would otherwise subclass just to reuse code.

WHY IT MATTERS

Inheritance reuse breaks down on two fronts: behavior that varies along several axes explodes into a lattice of subclasses, and the fragile base class means a harmless-looking parent edit silently breaks descendants that lean on its internals. Composition sidesteps both — abilities become small parts you mix at runtime, and collaborators are coupled only through their interfaces, so each can change alone.

— How it works

Component

A small, focused collaborator that owns one behavior (a MoveStrategy, a Logger). Reusable on its own.

Host

The object that holds components and delegates to them. It gains behavior by composition, not by inheriting it.

Interface

The narrow contract between host and component. The host depends on this, never on a concrete component's internals.

HOW TO APPLY

1. When tempted to subclass, ask: is this a true is-a, or am I just reusing code?
2. If it is reuse, extract the varying behavior into a component with a small interface.
3. Have the host hold the component and delegate to it.
4. Inject the concrete component from outside so it can be swapped or combined at runtime.

LITMUS TEST

Are you subclassing to reuse code, or to model a true is-a with a shared contract?
Reuse-only is a signal to compose instead.

— Smell → fix

Every combination of abilities needs its own subclass:

```
class Player {}  
class FlyingPlayer extends Player {}  
class SwimmingPlayer extends Player {}  
class FlyingSwimmingPlayer extends Player {} // explosion
```

↓ INJECT BEHAVIOR AS A COLLABORATOR

```
function makePlayer(move) { return { act() { move.go() } } } // has-a  
makePlayer(new FlyMove())  
makePlayer(new SwimMove()) // mix abilities at runtime, no subclass
```

Abilities become injected strategies. New combinations are wiring, not new classes — and the player no longer depends on any base class internals.

USE WHEN

- ✓ Behavior varies along multiple axes
- ✓ Avoiding deep, fragile hierarchies

AVOID WHEN

- ▲ A genuine, stable is-a relationship with a shared contract

— Trade-offs & pitfalls

PROS	CONS
+ Runtime flexibility + Avoids the fragile base class	– More objects and wiring – Indirection

COMMON PITFALLS

- ▲ Deep inheritance trees built for code reuse — the classic fragile-base-class trap.
- ▲ Swinging too far: composing trivial, never-varying behavior adds wiring for no gain.
- ▲ Calling it composition while the host still reaches into a concrete component instead of its interface.

WHERE YOU SEE IT

- React: a custom hook (has-a behavior) instead of a class-component inheritance chain.
- Strategy and Decorator patterns — both are composition over inheritance in action.
- Express or Nest middleware composed in a pipeline rather than subclassing a base handler.

SEE ALSO

Strategy	the pattern that realizes composition of behavior
Decorator	compose responsibilities by wrapping instead of subclassing
LSP	use inheritance only where substitutability truly holds
DIP	depend on the collaborator’s interface; inject the concrete one

— Interview & sources

Why "favor" composition rather than always compose?

Inheritance still fits a genuine is-a with a shared, stable contract. Composition wins when behavior varies or you would subclass purely to reuse code.

What is the fragile base class problem?

Subclasses depend on the parent's implementation details, so a safe-looking change in the base can break descendants in non-obvious ways.

How does this connect to the GoF?

They state it directly: "favor object composition over class inheritance." Most patterns lean on composition for exactly this flexibility.

When is inheritance still the right call?

For a genuine is-a with a stable, shared contract where substitutability holds (LSP) — e.g. extending a framework base class at one defined point. Reuse-only motivation is the red flag.

REMEMBER

Reuse behavior by holding objects (has-a), not by extending them (is-a) — unless it is a true is-a.

SOURCES

Gamma, Helm, Johnson, Vlissides — "Design Patterns" (1994): favor object composition over class inheritance
Joshua Bloch — "Effective Java": item "favor composition over inheritance"